

МИНОБРНАУКИ РОССИИ
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Балтийский государственный технический университет «ВОЕНМЕХ» им. Д.Ф. Устинова»
(БГТУ «ВОЕНМЕХ» им. Д.Ф. Устинова)

Кафедра	<u>О7</u>	<u>Информационные системы и программная инженерия</u>
	шифр	наименование кафедры, по которой выполняется работа
Дисциплина	<u>Технологии распространения, развертывания и сопровождения</u>	
	<u>программного обеспечения</u>	

Лабораторная работа
№2

номер задания (при наличии)

Способы представления и развертывания ПО. Зависимости ПО

Вариант №5

при наличии указать тему учебно-практической работы и (или) номер варианта

ОБУЧАЮЩИЙСЯ

группы О717Б

Праудин Д. С.

подпись

фамилия и инициалы

дата сдачи

ПРОВЕРИЛ

ученая степень, ученое звание, должность

Мажайцев Е.А.

подпись

фамилия и инициалы

Оценка / балльная оценка

дата проверки

Санкт-Петербург
2025 г.

1 Формулировка задания

Выполнение данной работы состоит из четырёх этапов:

- написание скриптов сборки, тестов, упаковки ПО;
- настройка `gitlab-ci.yml` файла;
- настройка сборочной линии (pipeline) и подключение `gitlab-runner`;
- сборка релизной версии после успешного прохождения пайплайна.

2 Ход выполнения задания

2.1 Написание скриптов стадий разработки

Всего в сборочной линии (pipeline) используется три стадии (stages) разработки: сборка, тестирование и упаковка. Для стадии тестирования был разработан специальный bash-скрипт, представленный ниже:

```
#!/bin/bash
run_test() {
    input="$1"
    expected="$2"
    result=$(echo -e "$input" | ./usr/bin/word-counter | sed '1d')
    if [[ "$result" == "$expected" ]]; then
        echo "✓ PASSED: '$input' -> $result"
    else
        echo "✗ FAILED: '$input' -> $result (expected: $expected)"
        exit 1
    fi
}

run_test "Hello world" "Количество слов: 2"
run_test "one two three" "Количество слов: 3"
run_test "" "Количество слов: 0"
run_test " spaced out words " "Количество слов: 3"
run_test "\tТабуляция\t\tДве табуляции Пробел" "Количество слов: 4"
echo "🎉 All tests passed!"
```

2.2 Настройка gitlab-ci.yml файла

Для описания работы pipeline был создан файл gitlab-ci с расширением .yml. В нём описываются стадии разработки и действия, которые необходимо выполнить на каждой стадии. Текст данного файла представлен ниже:

```
stages:
  - build
  - test
  - package

build:
  stage: build
  script:
    - make

test:
  stage: test
  script:
    - make test

package:
  stage: package
  script:
    - dpkg-buildpackage -us -uc
    - fakeroot debian/rules clean
    - mv ../word-counter*.deb .
  artifacts:
    paths:
      - word-counter_1.0_amd64.deb
```

2.3 Настройка сборочной линии. Подключение gitlab-runner

Для добавления сборочной линии сначала необходимо локально установить gitlab-runner с официального репозитория.

После того, как gitlab-runner был установлен, требуется перейти в настройки CI/CD текущего проекта на GitLab и создать новый runner. Соответствующая страница GitLab представлена на рисунке 1.

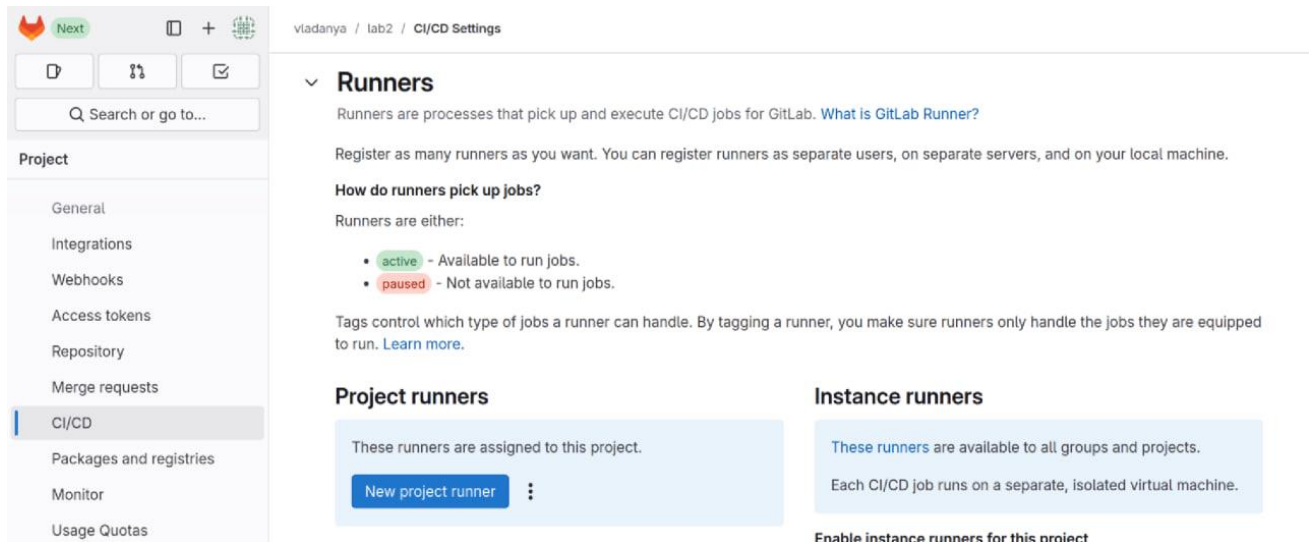


Рисунок 1 – Страница GitLab с добавлением Project runners

По нажатию на кнопку “New project runner” перед пользователем открывается страница с инструкцией для регистрации нового runner, что изображено на рисунке 2.

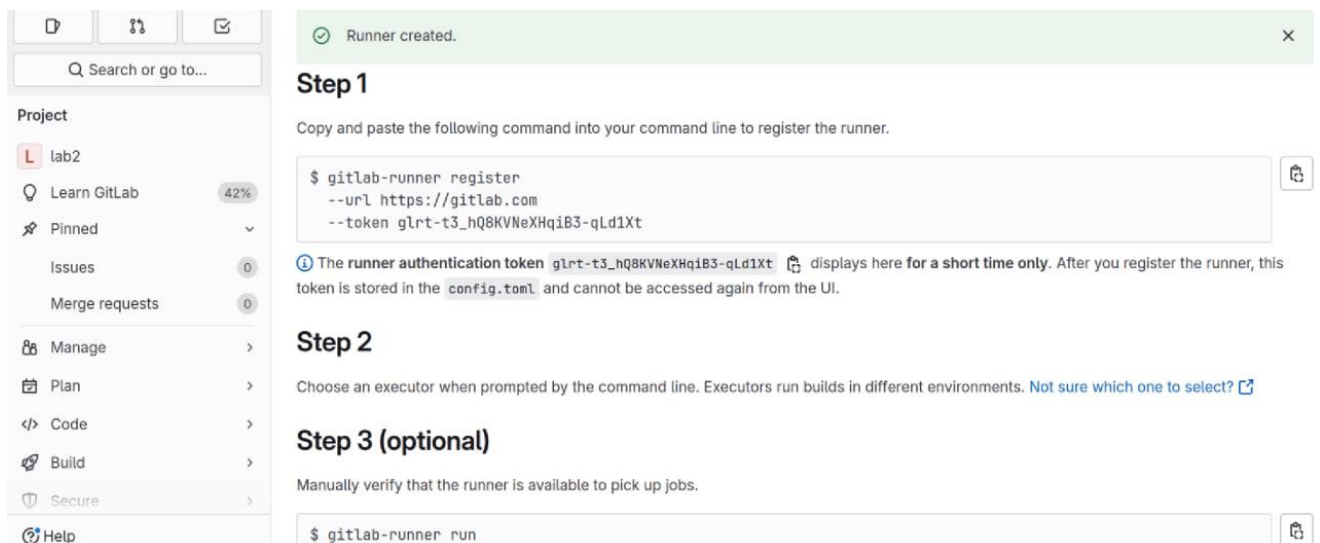


Рисунок 2 – Страница GitLab с инструкцией для регистрации runner

Сделав всё по инструкции, в терминале ОС появится информация о запущенном сервисе, которую можно наблюдать на рисунке 3. Другим

признаком работающего процесса gitlab-runner является обновление списка активных экземпляров для данного проекта, что продемонстрировано на рисунке 4.

```
vladislav@vladislav-VirtualBox:~/Desktop/ddm/lab1/word-counter-1.0$ gitlab-runner run
Runtime platform arch=amd64 os=linux pid=9251 revision=67b2b2db
version=17.10.0
Starting multi-runner from /home/vladislav/.gitlab-runner/config.toml... builds=0 max_builds=0
WARNING: Running in user-mode.
WARNING: Use sudo for system-mode:
WARNING: $ sudo gitlab-runner...

Usage logger disabled builds=0 max_builds=1
Configuration loaded builds=0 max_builds=1
listen_address not defined, metrics & debug endpoints disabled builds=0 max_builds=1
[session_server].listen_address not defined, session endpoints disabled builds=0 max_builds=1
Initializing executor providers builds=0 max_builds=1
```

Рисунок 3 – gitlab-runner запущен в терминале ОС

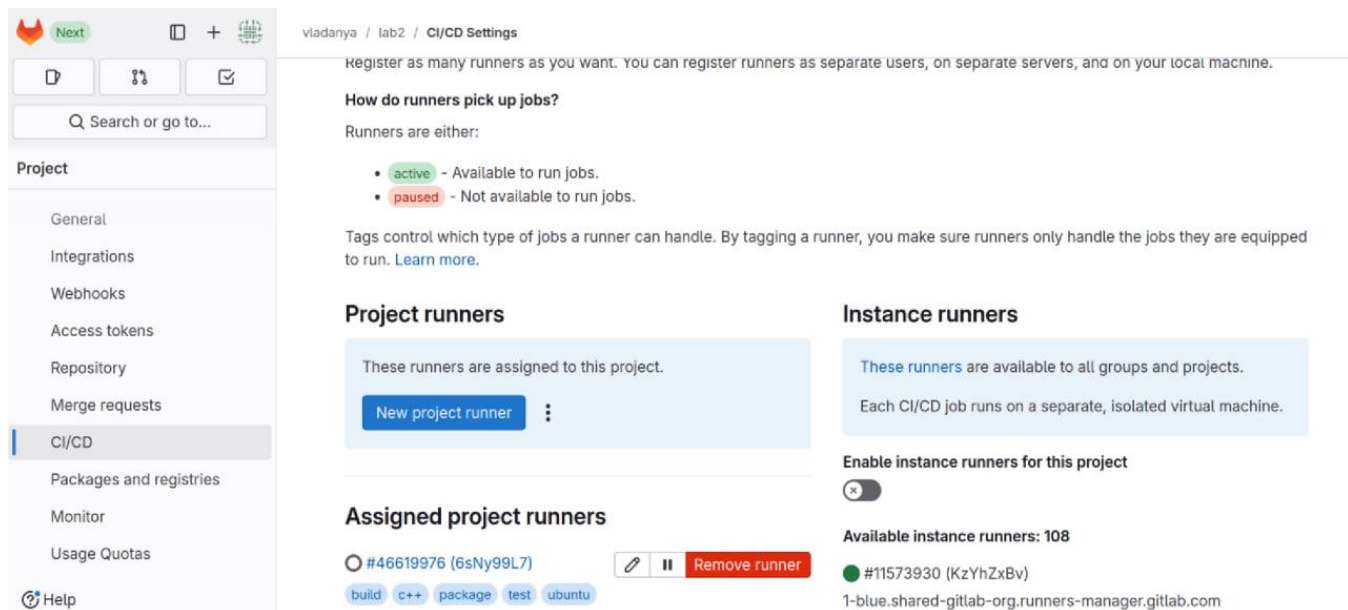


Рисунок 4 – Экземпляр gitlab-runner добавлен в список использующихся

Последним шагом перед использованием сборочной линии осталось лишь добавить новый pipeline, как показано на рисунке 5.

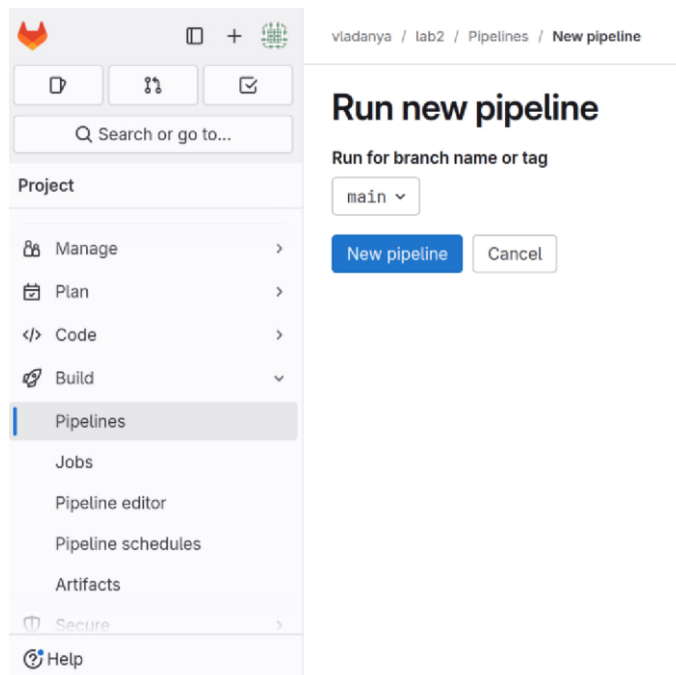


Рисунок 5 – Добавление новой сборочной линии

2.3 Прохождение pipeline и сборка релизной версии

После добавления новой сборочной линии и предварительной загрузки файла `gitlab-ci.yml` на GitLab у пользователя появляется возможность запускать сборочную линию и отслеживать процесс её прохождения в реальном времени. Пример того, как это выглядит продемонстрирован на рисунке 6.

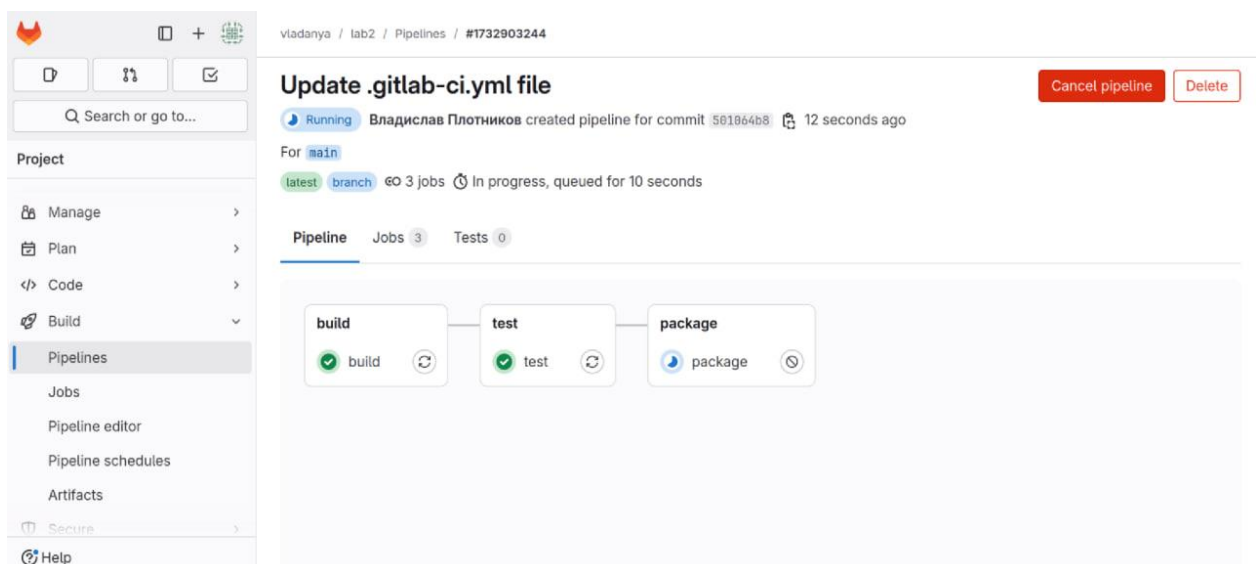


Рисунок 6 – Процесс прохождения сборочной линии

Для получения более детальной информации о каждой стадии можно ознакомиться с выводом онлайн-терминала, кликнув по соответствующей стадии, что представлено на рисунке 7.

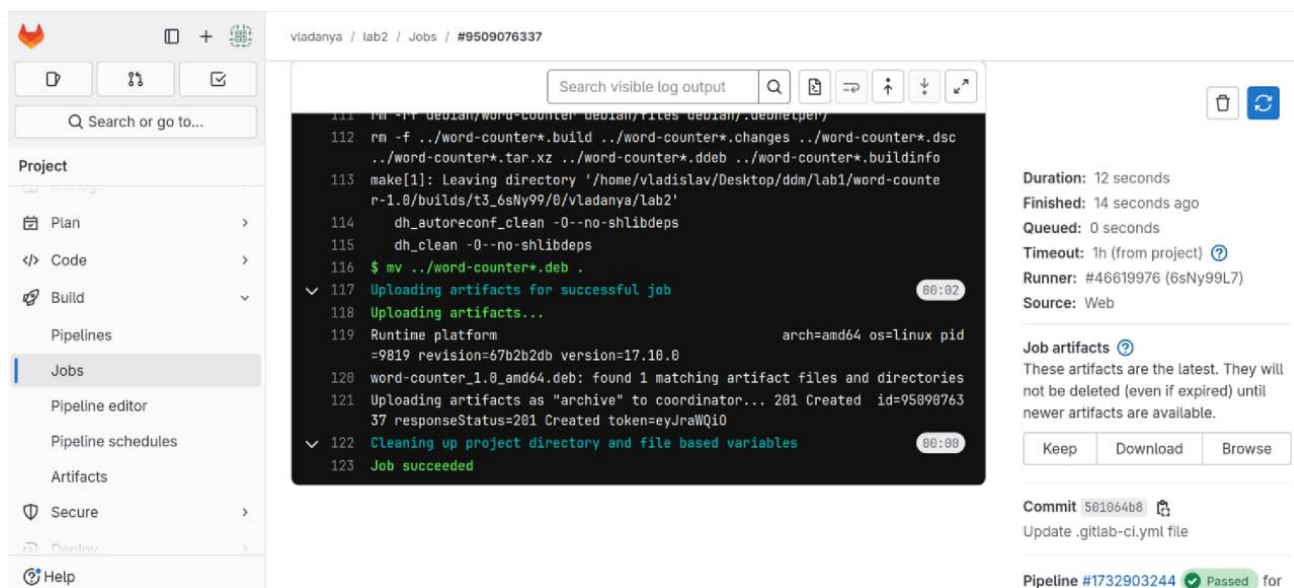


Рисунок 7 – Дополнительная информация о стадии упаковки ПО

Дождавшись успешного прохождения сборочной линии, пользователю предоставляется возможность скачать артефакт – релизную версию продукта. На рисунке 8 продемонстрирована данная возможность.

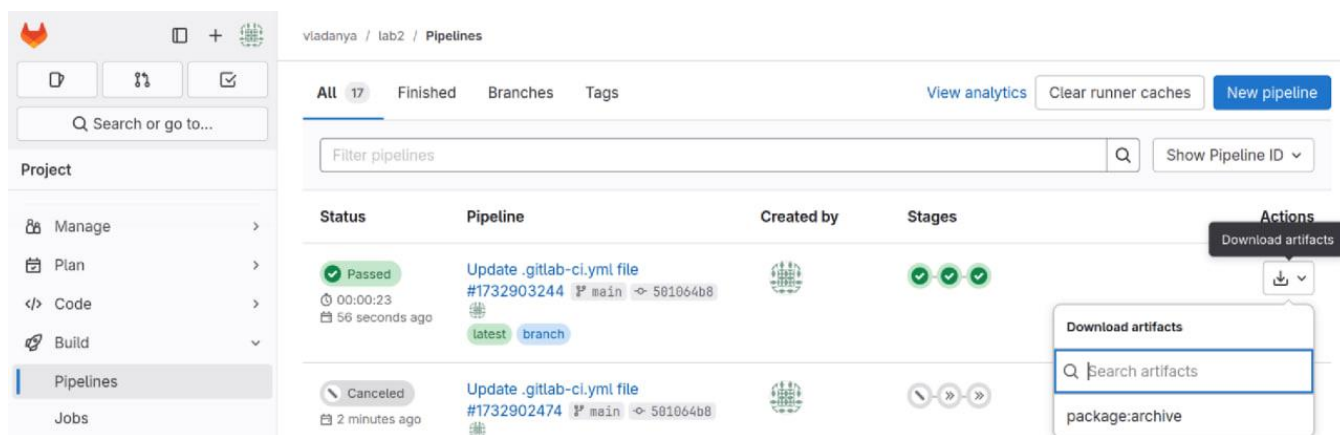


Рисунок 8 – Скачивание артефакта

В результате нажатия на кнопку скачивания происходит загрузка выбранного артефакта. Результат скачивания релизной версии разработанного ПО изображён на рисунке 9.

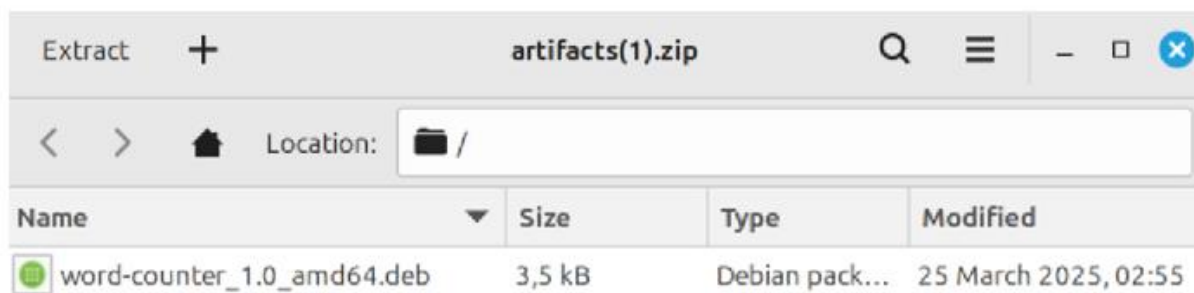


Рисунок 9 – Скачанный артефакт

ЗАКЛЮЧЕНИЕ

В результате выполнения работы был разработан скрипт для тестирования программы, загружен gitlab-runner и создан файл gitlab-ci для настройки сборочной линии, а также получен артефакт с релизной версией программы. Все поставленные задачи выполнены.